

Architectural Enhancement Report of Gemini CLI

Apr 6, 2026

Authors:

Mehrin Hossain (23nt1@queensu.ca)
Lauren Sutherland (21las46@queensu.ca)
Jaineel Modi (22rsw2@queensu.ca)
Sophia Kim (18yjk@queensu.ca)
Minh Quang Tran (22rfx@queensu.ca)
Michael Okene (22gvl2@queensu.ca)

1 Introduction.....	2
1.1 Motivation.....	2
2 New Feature Overview.....	3
3 Implementation Approaches.....	3
3.1 Localized Worktrees with Extended Remote Capabilities:.....	3
3.2 Containerized Worktree Isolation.....	4
4 SAAM Analysis.....	6
4.1 Major Stakeholders and their Important NFRs.....	6
4.2 Best Approach to Realize the Enhancement.....	7
5 Impact on Architecture.....	7
5.1 Impact of Localized Worktrees Implementation.....	7
5.2 Impact of Containerized Worktree Implementation.....	8
6 Potential Risks.....	9
7 Testing.....	10
8 Use Cases.....	11
8.1 Use Case 1: Push a Synced Worktree Session to Remote.....	11
8.2 Use Case 2: Pull and Resume a Synced Worktree Session.....	12
9 Conclusion.....	13
10 Lessons Learned.....	14
11 References.....	14
12 AI Report.....	15

Abstract

Currently, Gemini CLI has been shifting towards becoming an all-encompassing project manager that supports more concurrent and task-oriented subagent execution using native Git worktrees. Our findings have shown that this developing feature lacks any mention of support for extensible remote sessions across multiple devices, especially for parallel task execution via subagent-run Git worktrees. Therefore, the following enhancement proposal for Gemini CLI will introduce remote worktree session synchronization to enable developers to access worktree sessions across different devices.

In order to implement remote worktrees within Gemini CLI, this report offers two architectural alternatives: Localized Worktrees with Extended Remote Capabilities and Containerized Worktree Isolation. To choose the best alternative for the proposed enhancement, a SAAM will be carried out by taking into account major stakeholders and their needs as well as several non-functional requirements such as maintainability, evolvability, testability, performance, usability, reliability, and security of the system.

Based on the above mentioned SAAM analysis, localized worktrees seem to be the most appropriate choice due to its reuse and extensibility in regard to Gemini CLI worktrees and Git execution capabilities. However, there are still some potential risks associated with the proposed enhancement which may appear due to remote sessions, state management, restoration, as well as security concerns.

1 Introduction

Gemini CLI is an AI-enabled development environment that uses a combination of command-line interface, agent-based orchestration, and tools execution. The current trend in the development of Gemini CLI involves a shift towards a multi-task approach utilizing Git worktrees and Directed Acyclic Graphs (DAG) for storing execution state information. Parallel execution, proper isolation of tasks, and improved code management by keeping changes separate using Git worktree functionality will be beneficial.

One of the features currently implemented in Gemini CLI is the use of Git worktrees. As many branches can be created in separate directory locations from a single repository, worktrees help isolate individual tasks from each other and prevent filesystem-related conflicts in executing them simultaneously. Currently, there is a limited set of worktree capabilities available in Gemini CLI, and some of them include support for more advanced task management, convenient switching between tasks, automatic cleanup, and improved support for the task's lifecycle management. But the most urgent problem at the moment is continuation of a specific task in a different device.

This report highlights the need for this constraint and discusses an improvement that implements remote session synchronization capability for Git worktrees. This enables a programmer to upload their ongoing session to remote servers, enabling them to resume the same task on a new machine with the associated branch status, minimized contextual information, and task metadata still in place. From a design perspective, this shifts the management of worktree tasks from being a locally scoped and session-based process into one that can persist across different devices. This improvement is useful since it facilitates continuous development across multiple machines, provides better continuity of tasks, and makes resuming complicated tasks less disruptive.

Two possible implementations for this improvement have been proposed in this report and have been evaluated based on SAAM by considering relevant non-functional requirements specific to different stakeholders. Their effects on system architecture, potential threats, testing considerations, and applications have been analyzed in detail. This report concludes that remote worktree extension is superior compared to a containerized execution paradigm since it is a better fit for Gemini CLI's current system architecture.

1.1 Motivation

In terms of motivation for the proposed enhancement, it appears clearly as a consequence of a certain gap in the present trajectory of Gemini CLI development. On the one hand, the system is becoming increasingly better at handling parallel, task-specific work using Git worktrees. Yet on the other hand, the capabilities of Gemini CLI appear to be limited by the current proximity of task states to the particular device and a particular session of active work. Once a programmer stops the development session on his or her computer, changes environments, or goes to work from a different device altogether, the continuity of this work coordination will become compromised.

This problem can be fixed by creating remote sessions of tasks, which will involve keeping branch state, compressed agent context, and DAG structure describing task metadata not just in the computer's RAM during an active session but also in some remote location from which it will later be possible to restore the session. This will allow a programmer to resume the work not with only the knowledge of what he or she had been doing, but with all the context intact.

The improvement is valuable architecturally. The improvement makes Gemini CLI move even further along the path of developing its approach to handling projects as independent tasks to be accomplished, as opposed to a simple set of files and directories to be managed locally. The improvement enhances usability, minimizes interruption overhead, and facilitates future expansion, including but not

limited to distributed workflow management. For these reasons, worktree synchronization with a remote system is more than a convenience option. It is an architectural advancement of the nascent Gemini CLI project coordination paradigm.

2 New Feature Overview

With a broader aim to shift Gemini CLI from a single-session, linear AI assistant to a multi-threaded concurrent, autonomous project manager, Gemini CLI is in the midst of implementing a restructured framework that involves the separation of Plans (intent) vs. Trackers (execution state) as nodes within a Directed Acyclic Graph (DAG) to manage safer, parallel task execution using Git worktrees [5].

A key feature of this new overarching framework is to use Git's native worktree feature, which allows multiple branches from the same repository to each have a separate, isolated physical folder [3]. This allows for subagents to work on parallel tasks without the risk of filesystem collisions from overwriting, and acts as a sandbox for isolating code changes to preserve the main dev environment. Currently, this is a partially implemented, experimental feature with the latest changes introducing Git worktree support using the `--worktree` (or `-w`) flag to create an isolated workspace and automatically handling worktree and branch creation without the need for manual Git commands [1]. Gemini CLI prioritizes a safety-first design, which means that it will preserve the worktree upon exit if any uncommitted changes or new commits are detected [1]. Some aspects of this feature have yet to be implemented, but they are currently in development. These aspects include the use of a `/worktree` command for interactive context switching after the session has begun [7], Git worktree integration with tasks (including automated merging and/or collision handling) and subagents, automatic environment initialization for new worktrees, and automated cleanup or archival of completed tasks [5]. The proposed enhancement extends this feature further by suggesting the incorporation of remote session syncing for multi-device support and optimization.

This enhancement aims to address the current limitations of session management for Git worktrees, which do not allow for resuming or accessing session contexts, uncommitted changes, and task states across multiple devices. Allowing remote session syncing across various devices allows for continuous development and enables task-focused workspaces. In addition, our approaches allow for faster context switching and resumption time for remote isolation, introduces more space efficiency through the utilization of tactical compression, and simplifies the Git worktree lifecycle management for developers by handling explicit Git calls and wrapping it into automated commands.

3 Implementation Approaches

3.1 Localized Worktrees with Extended Remote Capabilities:

This enhancement bridges a fundamental gap in Gemini CLI's current architecture: the multi-agent project manager can orchestrate parallel tasks beautifully within a single session, but the moment you close your laptop or switch machines, that entire coordination state evaporates. Remote session syncing extends the lifecycle of every worktree from in-session memory to persistent, cross-device project state. Gemini CLI's multi-agent system has a clear hierarchy: a parent orchestrator agent holds the task graph (the DAG), spawns child agents into isolated git worktrees per task [6], and tracks which branches map to which task IDs [1]. Currently, all of that lives in process memory and a local `.gemini/` directory [2].

This enhancement inserts a worktree orchestration engine as a middleware layer between the parent agent and the filesystem [10]. It intercepts two new command surfaces: `--worktree push` and `--worktree pull <taskID>` and handles the serialization, transport, and hydration of everything the parent agent needs to resume coordination on any device. The `.gemini/config.json` stores the remote backend selector (which storage system to use), the `project_hash` used as a namespace in remote storage, and a `taskMap` that keeps local worktree paths in sync with remote task IDs [5]. When the parent agent needs to know the status of a task, it reads this config first. If the worktree is local, it reads from `.gemini/worktrees/<taskID>`; if it is remote, it hydrates from whichever backend is configured [1].

Remote backend options like OpenTelemetry backends (Jaeger, Tempo, or any OTel-compatible collector), with a possible extensible state store database, are a natural fit because DAG state is structurally similar to distributed trace data. Specifically, tasks have parent-child spans, durations, status codes, and baggage [5]. The tracker JSON maps cleanly onto span attributes.

When a developer runs `--worktree push` inside an active worktree session, the orchestration engine executes a sequence that the parent agent treats as a state transition [8]. First, it commits and pushes all staged changes to the task's associated git branch using standard git commands (`add`, `commit`, `push`) [3]. Second, it calls Gemini CLI's built-in `/compress` command on the current chat context, producing a dense summary that preserves the reasoning, decisions, and in-progress work without carrying the full token history [2]. Third, it uploads this compressed context blob to the configured remote backend, keyed by taskID so any device that knows the task ID can retrieve exactly this context snapshot. Fourth, it serializes the current DAG node state (task status, dependencies satisfied, subtask outputs) to the remote tracker and sends an acknowledgment back to the parent agent, which updates its in-memory DAG and writes the new taskMap entry to config [5].

`--worktree pull <taskID>` begins by fetching the DAG state, task metadata, and dependency graph for that task from the remote tracker [5]. This tells the orchestration engine what branch to check out, what subtasks are complete, and what the agent was working toward. It then runs `git worktree add .gemini/worktrees/<taskID> <branch>` to place the existing task branch into a fresh worktree path. Importantly, this is a worktree reference to an existing branch, so the parent agent can still track it as part of the same DAG [3]. For monorepo setups, the pull command optionally activates git sparse checkout at this point [4]. Rather than materializing the entire repo into the worktree, it loads only the file patterns relevant to this task. This is possible because the DAG state encodes which subdirectories each task node touches [4][7]. This keeps worktree restore time fast even in large repositories. The compressed context blob is retrieved from remote storage and written to `~/.gemini/tmp/<project_hash>/chats/`, exactly where Gemini CLI expects to find chat history for a given worktree session [1]. When the child agent spins up in this worktree, it loads that compressed context as its starting state, meaning it knows what was tried, what was decided, and where work stopped [7]. Finally, the environment is hydrated: `node_modules` is symlinked from the project root into the new worktree (`ln -s /path/path/to/symlink`), assuming npm install was run at the repo root at some point [1]. This avoids re-running installs for each worktree and is the same approach Gemini CLI uses for its own monorepo worktree setup [1]. The restored worktree is then registered with the parent agent, which reactivates the DAG node and marks the task as `ACTIVE_LOCAL` again on the new device [6].

3.2 Containerized Worktree Isolation

Each worktree is backed by its own Docker/Podman container [11]. The container owns the build environment and execution layer; the worktree directory on the host is bind-mounted into the container as the only writable surface [12]. A push/pull wrapper syncs both the Git state and the compressed agent context to a shared remote, allowing any device with the CLI installed to restore a full working session [1][3].

At its core, the implementation introduces a container management layer that sits between the Gemini CLI agent process and the host filesystem. Rather than allowing the agent to execute tool calls directly, every tool call is intercepted by a container manager that routes it through a Unix domain socket to a dedicated container for that task [15]. The agent itself never knows whether it is talking to a local shell or a remote container. The interface is identical from the agent's perspective, meaning the containerization layer is entirely transparent to the existing agent logic and requires no changes to the agent's reasoning or plans [10].

Each container is structured around four image layers with different cache lifetimes [13][14]. The base layer contains a minimal OS, Git, the agent runner binary, and the socket server, which is pulled once and almost never changes. Above it sits a runtime layer built from the project's lockfile, cached by lockfile hash so that a container can be started quickly when dependencies have not changed [14]. A project snapshot layer, built once per session from the base commit, is shared read-only across every container spawned in that session, meaning parallel tasks never duplicate the full source tree [12]. The top layer is unique per container; however, agent writes never actually land in this layer because they go into a bind-mounted worktree directory on the host filesystem [12]. This means teardown is a clean docker rm with no container-layer residue, and the diff is always a standard git diff rather than a container image diff [3][11].

The cross-device sync capability is built on top of this isolation foundation through two coordinated data flows. When a developer runs `-w push`, the system commits the worktree's Git changes to a task branch on the shared remote, compresses the agent's conversation context down to a structured summary block, stripping raw tool outputs and retaining only decisions and outcomes, then uploads it to a configured remote backend keyed by task ID [2][5]. The tracker state, a DAG node carrying the branch ref, base commit, files touched, dependency manifest, and last push metadata, is written to the same remote [5]. This design deliberately separates three concerns that are often conflated: the code state lives in Git, the environment state lives in the image layer cache, and the agent context lives in the compressed summary store [8]. Each travels through its own appropriate channel rather than being bundled into a single opaque artifact [10].

On the receiving device, `-w pull <taskID>` reverses this process in strict sequence. The tracker state is fetched first to validate that the task branch actually exists on the remote before any local filesystem changes are made [5]. The worktree is then recreated via `git worktree add` pointing at the task branch, with optional sparse checkout applied for monorepos so only the directories the task actually touched are materialized [3][4]. The compressed context is downloaded, hash-verified, and written to the local chat cache directory where it will be injected as a prior context prefix the moment the agent session starts, giving the agent on Device B full awareness of everything that was decided and done on Device A without requiring the raw conversation history [1][7]. Finally, the build environment is hydrated: if the lockfile hash in the tracker matches what is present at the project root, `node_modules` is symlinked rather than reinstalled, which reduces restore time from minutes to seconds [1]. A fresh container is then spawned against the restored worktree with the same three-binding mount strategy, namely a read-only project root, a read-write worktree, and a Unix socket [12][15], and the agent connects and resumes as if the session had never moved [6].

4 SAAM Analysis

4.1 Major Stakeholders and their Important NFRs

The proposed enhancement introduces remote synchronization of Git worktree sessions, which enables isolated task workspaces, DAG state, and compressed agent context to persist across multiple devices [1][3]. Because this enhancement changes how execution state is stored, restored, and coordinated, the primary stakeholders include Gemini CLI core developers, end-user software developers, and infrastructure/security maintainers. The two implementation approaches considered are Localized Worktrees with Extended Remote Capabilities and Containerized Worktree Isolation. For Gemini CLI core developers, the most important non-functional requirements are maintainability, evolvability, and testability [8][9][10]. Maintainability is critical because the enhancement must integrate with the existing Orchestration Engine, Tool Execution Framework, and Configuration System without introducing excessive architectural drift [2][10]. The localized worktree approach better supports maintainability because it extends existing Git-native worktree semantics and reuses current TEF tool pathways for Git commands, task mapping, and configuration persistence [1][2][3]. In contrast, the containerized approach introduces a container management layer, image cache lifecycle, socket routing, and bind-mount orchestration, which increases complexity in the subsystems and makes future modifications more difficult [10].

Evolvability is also a major concern, as this enhancement is likely to serve as the foundation for future capabilities such as collaborative multi-user DAG execution, cloud-based task restoration, and persistent long-running subagents [10]. The localized worktree approach provides stronger evolvability because the remote DAG tracker and synchronization metadata remain backend-agnostic, allowing future extensions without requiring major structural redesign. While the containerized approach is also extensible, its dependency on container runtime policies and environment image layers makes future architectural changes more invasive [10]. From a testability perspective, the localized worktree approach again provides the stronger option [8][9]. Since the workflow primarily consists of DAG serialization, remote metadata transfer, worktree restoration, and configuration hydration, the enhancement can be validated through deterministic unit and integration tests. By comparison, the containerized approach significantly expands the regression surface by requiring validation of container startup correctness, image layer reuse, Unix socket communication, bind mount consistency, and teardown behavior across devices. For end-user software developers, the most important NFRs are performance, usability, and reliability [9][10]. Performance is especially important because task restoration must feel fast enough to support seamless context switching between devices. The localized worktree approach provides strong performance through sparse checkout, compressed context summaries, Git-native branch restoration, and dependency symlinking, minimizing restore latency [3][4]. The containerized approach offers consistent execution performance after startup, but cold-start delays from container bootstrapping and image hydration can negatively affect perceived responsiveness.

Usability focuses on how naturally the enhancement fits into existing developer workflows. The localized worktree approach better aligns with current Git mental models through intuitive push/pull semantics and preserved filesystem transparency [1][3]. The containerized approach introduces an additional abstraction boundary that may obscure logs, file visibility, and runtime diagnostics, slightly increasing the learning curve for end users. Reliability is equally important because task state must remain correct during device migration [10]. The localized worktree approach depends on accurate tracker serialization and remote consistency guarantees, while the containerized approach benefits from reproducible execution environments that reduce host-machine inconsistencies. However, this reliability gain comes at the cost of additional architectural overhead. For infrastructure and security maintainers, the most important NFR is security [10]. The localized worktree approach introduces moderate risk because DAG state, compressed chat context, and task metadata are persisted remotely, increasing

exposure to unauthorized restoration or stale task replay if backend validation is weak. The containerized approach provides stronger isolation guarantees because tool execution remains constrained to dedicated task-local containers, reducing filesystem exposure and improving sandbox boundaries.

4.2 Best Approach to Realize the Enhancement

After evaluating both approaches against stakeholder-specific non-functional requirements, Localized Worktrees with Extended Remote Capabilities is the stronger architectural choice for realizing the enhancement [8][9][10]. Although the containerized worktree isolation approach provides stronger runtime isolation and improved execution reproducibility, its architectural cost is significantly higher in terms of maintainability, testability, and subsystem coupling [10]. Introducing container lifecycle management, image caching policies, socket routing, and bind-mount orchestration adds substantial complexity across the Application, Service, and Infrastructure layers..

The localized worktree approach aligns more naturally with Gemini CLI's current concrete architecture due to it directly extending the existing Git worktree support, DAG-based task coordination, Tool Execution Framework Git command paths, and configuration persistence mechanisms [1][2][3]. This reuse minimizes disruption to current subsystem responsibilities while still enabling significant new functionality. Most importantly, this approach provides the strongest balance across the major architectural qualities required by the enhancement [8][9][10]. It preserves maintainability through subsystem reuse, supports evolvability for future distributed workflows, improves testability through deterministic synchronization flows, offers strong performance through fast Git-native restoration [3][4], and maintains a highly usable developer workflow based on familiar Git semantics [1][3]. While its security posture is weaker than the containerized alternative, the risks can be sufficiently mitigated through backend authentication, encrypted metadata storage, and tracker validation [10]. For these reasons, the localized worktree approach offers the best overall balance of maintainability, evolvability, testability, performance, usability, reliability, and architectural fit.

5 Impact on Architecture

5.1 Impact of Localized Worktrees Implementation

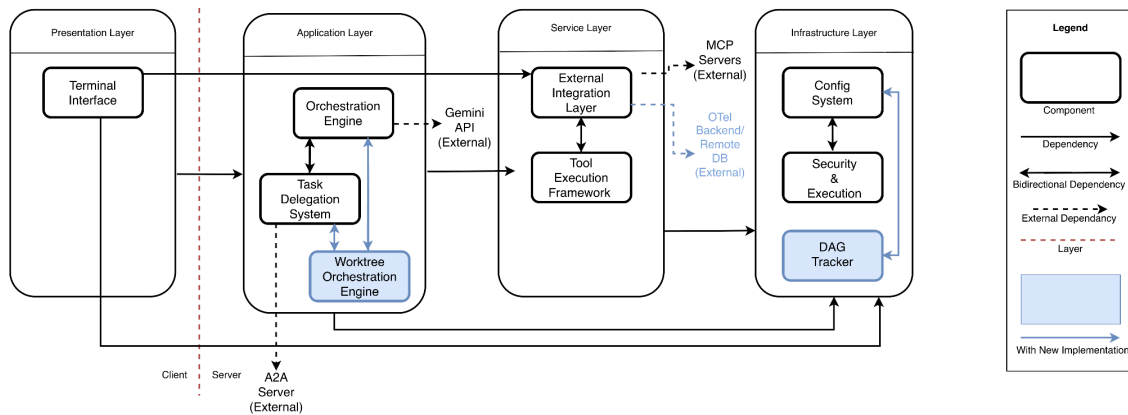


Figure 1: Concrete Architecture Diagram with Localized Worktrees Implementation

Impact on Subsystems

For the first implementation, the Orchestration Engine continues to coordinate task execution while interacting with the Task Delegation System and the newly introduced Worktree Orchestration Engine to manage worktree lifecycle and task-to-worktree mapping within the Application Layer. The Worktree Orchestration Engine handles creation, restoration, and synchronization triggers for worktrees without altering existing orchestration logic. The Tool Execution Framework remains responsible for executing commands but is extended to support Git-based operations required for worktree management. The External Integration Layer manages communication with external systems, including MCP servers and external backends such as OTEL-compatible storage, rather than introducing a separate connector subsystem. In the Infrastructure Layer, the DAG Tracker is introduced as the persistent state component responsible for storing task structure, dependencies, and execution status, interacting with both the Security & Execution component for controlled access and the Config System for retrieving backend configuration and task mappings. The Configuration System is extended to store task-to-worktree mappings and remote backend settings, while other subsystems such as the Task Delegation System and Security & Execution retain their core responsibilities with only minor extensions to support synchronization and persistence.

High-Level Concrete Impacts

A Worktree Orchestration Engine is introduced in the Application Layer, extending coordination capabilities without altering the core orchestration logic. The Service Layer is not expanded with new subsystems; instead, the External Integration Layer handles all external communication, including interaction with MCP servers and external storage backends.

The Infrastructure Layer introduces the DAG Tracker and extends the Configuration System, shifting task state from in-memory storage to a persistent, externally backed model. This shifts coordination from local to a distributed one, while preserving the existing layered architecture and clear separation of concerns.

Low-Level Concrete Impacts

The Orchestration Engine coordinates task execution through the Task Delegation System and interacts with the Worktree Orchestration Engine for worktree lifecycle operations. The Tool Execution Framework performs Git-based commands for worktree creation, restoration, and updates. The External Integration Layer manages communication with external systems like MCP servers and OTEL-compatible backends for state persistence and retrieval. The DAG Tracker stores and retrieves task state, while the Configuration System maintains task mappings and backend configuration.

5.2 Impact of Containerized Worktree Implementation

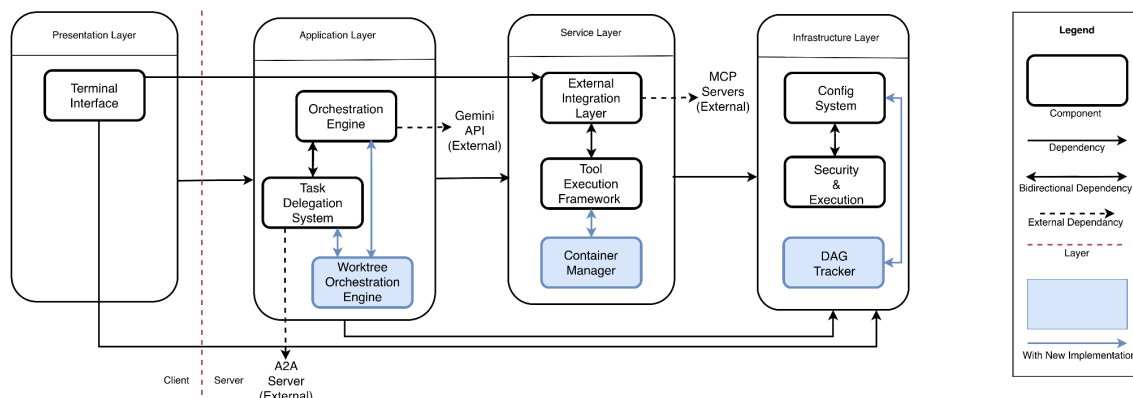


Figure 2: Concrete Architecture Diagram with Containerized Worktree Implementation

Impact on Subsystems

For the second implementation, the primary architectural change occurs within the Service Layer, where a new Container Manager subsystem is introduced to handle execution isolation. The Tool Execution Framework is modified to delegate all tool execution through the Container Manager rather than executing directly on the host system, allowing commands to be routed into isolated container environments. The Container Manager is responsible for managing container lifecycle, routing execution requests, and maintaining isolation boundaries while remaining transparent to higher-level orchestration logic. The Security & Execution component continues to govern execution control and works alongside the Container Manager to enforce secure execution. The Worktree Orchestration Engine, DAG Tracker, External Integration Layer, and Configuration System retain their roles from Implementation 1, with the Configuration System additionally supporting mappings between tasks and their associated containerized environments. Other subsystems, including the Orchestration Engine and Task Delegation System, remain structurally unchanged, with their responsibilities unaffected aside from interacting with the updated execution pathway.

High-Level Concrete Impacts

The architecture modifies the execution model within the Service Layer by introducing containerized execution as part of the existing Tool Execution Framework rather than adding a new subsystem. This creates an additional abstraction in how execution is performed, while preserving the orchestration and coordination structure defined in the first implementation. The rest of the architecture remains consistent, including the use of the External Integration Layer for external communication and the DAG Tracker for persistent task state, maintaining the distributed coordination model.

Low-Level Concrete Impacts

All tool execution is performed within isolated container environments managed through the execution pathway of the Tool Execution Framework. Worktrees are bind-mounted into these environments, ensuring consistency between the filesystem and execution context. The Tool Execution Framework delegates execution to the container runtime, while the DAG Tracker and Configuration System continue to manage task state and mappings. This introduces additional runtime overhead and dependency on container infrastructure but improves execution isolation and reproducibility without modifying higher-level coordination logic.

6 Potential Risks

The proposed enhancement introduces several architectural risks as Gemini CLI transitions from local ephemeral worktree state to distributed persistent synchronization across devices [1][3]. The most significant risk is security [2][10]. Because compressed agent context, DAG state, branch references, and task metadata are now stored in a remote backend, the system becomes vulnerable to unauthorized task restoration, stale task replay attacks, and exposure of sensitive project summaries [2][10]. If backend authentication or task ownership validation is weak, an attacker may be able to restore another user's task state or access compressed reasoning context [10]. This risk becomes more severe when third-party storage providers are used as synchronization backends [10].

A second major risk involves maintainability and architectural complexity [8][9][10]. The introduction of the Worktree Orchestration Engine, External Integration Layer, DAG Tracker persistence logic, and taskMap synchronization creates stronger coupling between the Application, Service, and Infrastructure layers [9][10]. Over time, these dependencies may erode the clean layered separation

established in the conceptual and concrete architecture reports, making future subsystem External Integration Layer evolution more difficult [8][9][10].

The enhancement also introduces performance risks due to its dependency on remote hydration workflows [3][4][10]. Network latency, backend response variability, large monorepo sparse checkouts, and dependency rehydration may increase task restoration time, especially for large repositories with complex task graphs [3][4]. If restore latency becomes too high, the usability benefits of seamless cross-device continuation may be reduced [10].

Another major concern is reliability and state consistency. Since Git branch state, remote DAG tracker state, and compressed task context now exist as separate synchronized artifacts, inconsistencies may emerge between them [1][3][10]. For example, a branch may contain newer commits while the remote tracker still reflects an older dependency state. This divergence could restore outdated reasoning context and cause the orchestration engine to resume an invalid task path [10].

Finally, the enhancement increases testability and validation risk. Cross device synchronization failures may depend on backend outages, interrupted uploads, partial DAG serialization, operating system path differences, or Git version inconsistencies. These distributed failure conditions are significantly harder to reproduce than the local-only workflows supported by the current system and therefore expand the regression surface of the overall architecture [8][9][10].

7 Testing

Because Localized Worktrees with Extended Remote Capabilities is the selected implementation approach, testing should focus on how remote session syncing interacts with existing Gemini CLI features rather than treating synchronization as an isolated function. The testing strategy should therefore combine unit tests, integration tests, and end-to-end cross-device tests. Unit tests can validate tracker serialization, configuration updates, and context metadata handling in isolation. Integration tests can use temporary repositories and a mocked remote backend to validate subsystem interactions. End-to-end tests should simulate a real developer workflow across two devices to confirm that a pushed task can be resumed correctly without breaking existing Gemini CLI behavior. Gemini CLI already documents experimental Git worktree support and the `--worktree` / `-w` workflow, making the current worktree lifecycle an appropriate baseline for these tests [1][2].

First, testing should verify interaction with the existing Git worktree lifecycle. A task should be created in a local worktree, modified, and then pushed using `-w push <taskID>`. The test should confirm that the correct branch is committed and pushed, that the local worktree remains isolated from the main repository, and that the remote tracker is only written after both the branch state and compressed context have been stored. The corresponding pull test should run `-w pull <taskID>` on a second device and confirm that the restored worktree points to the same task branch, appears at the expected local path, and does not overwrite unrelated local worktrees. This is consistent with Gemini CLI's current experimental worktree workflow and Git's linked-worktree model [1][3].

Second, testing should verify interaction with the DAG-based task coordination feature. For tasks with dependencies or subtasks, the pushed tracker should preserve task status, dependency satisfaction, and subtask outputs. After pulling on another device, the parent orchestrator should reconstruct the same execution state and reactivate the correct DAG node. Additional failure-injection tests should check stale tracker state, missing dependencies, or branch/tracker mismatches to ensure that the system rejects inconsistent restores instead of resuming an invalid task path.

Third, testing should verify interaction with the Configuration System and task mapping persistence. After a push, the configuration file should reflect the correct task-to-branch and task-to-remote mappings. After a pull, the new device should update the task map to the restored local worktree path without duplicating or corrupting prior entries. Restart tests should confirm that Gemini CLI can be closed and reopened after a push or pull while still recognizing the correct local or remote task status.

Fourth, testing should verify interaction with compressed context restoration. A task should be pushed after meaningful agent interaction so that the compressed summary contains decisions, intermediate reasoning, and current intent. After pulling, the restored session should load the correct compressed context for that specific task and allow work to continue without losing the task's prior direction. Negative tests should also verify that context blobs are not attached to the wrong task ID and that corrupted or incomplete context payloads are detected before restoration proceeds.

Fifth, testing should verify interaction with the Tool Execution Framework and environment setup. Integration tests should validate the Git command path used for push and pull, including commit, push, git worktree add, and optional sparse checkout for monorepos. Git explicitly documents git worktree add as the mechanism for creating linked worktrees and git sparse-checkout as a way to materialize only a subset of tracked paths, making both reasonable integration-test targets in this design [3][4]. These tests should also confirm that restored worktrees remain executable and that dependency hydration does not unexpectedly alter the original project root.

Finally, testing should verify backward compatibility and failure handling. Existing local-only workflows should continue to work when remote syncing is not used. The system should also be tested under backend outages, interrupted uploads, incomplete tracker writes, missing branches, and operating-system path differences. In all such cases, Gemini CLI should fail safely by surfacing a clear error, avoiding partial restoration, and preserving recoverable local state whenever possible. Together, these tests make the proposed enhancement feasible to validate and ensure that it interacts correctly with existing worktree isolation, DAG coordination, configuration persistence, context restoration, and execution behavior.

8 Use Cases

Legend:

- **Solid message arrow:** request, command, or operation being issued between subsystems
- **Dashed message arrow:** returned reference, state, context, or summary
- **Vertical dashed lines (lifelines):** each participant's timeline; time flows top to bottom

8.1 Use Case 1: Push a Synced Worktree Session to Remote

Use Case Summary: A developer asks Gemini CLI to sync an active worktree session by running `-w push <taskID>`. The Terminal Interface loads context from the Config System and submits the push request to the Orchestration Engine. The Orchestration Engine delegates the workflow to the Worktree Orchestration Engine, which first pushes the task branch through the Tool Execution Framework to the Git Remote. After the branch reference is returned, the Worktree Orchestration Engine compresses the task context and uploads it through the External Integration Layer. It then retrieves the current task state from the DAG Tracker and writes the remote tracker entry containing the branch reference, context

reference, and task state. Once the tracker is acknowledged, the Orchestration Engine returns a summary to the Terminal Interface, which displays the result to the user.

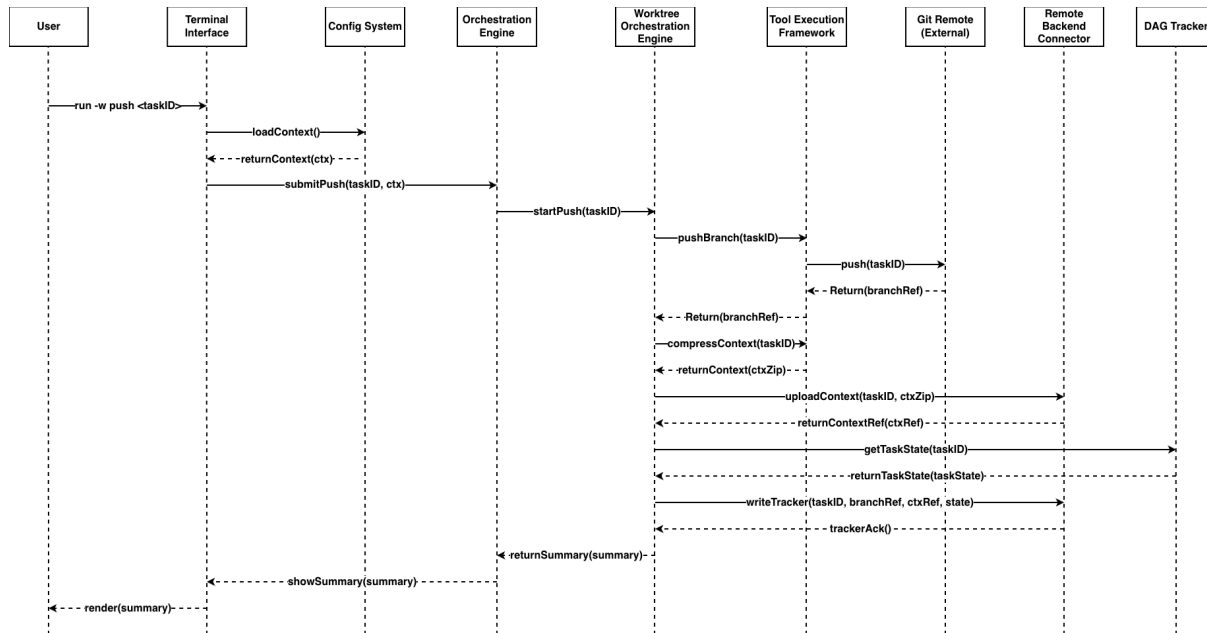


Figure 3: *Sequence Diagram for Use Case #1 (Push Synced Worktree Session to Remote)*

Explanation: This use case shows the concrete A3 workflow for synchronizing an in-progress task across devices. The Terminal Interface receives the push command, loads project context from the Config System, and forwards the request to the Orchestration Engine. The Orchestration Engine then invokes the Worktree Orchestration Engine, which coordinates the full sync process. First, it pushes the task branch to the Git Remote through the Tool Execution Framework. Second, it compresses and uploads the task context through the External Integration Layer Third, it retrieves the latest task state from the DAG Tracker and writes the tracker entry remotely. This ordering is important because it ensures that the tracker is only written after both the code state and the compressed context already exist remotely. Compared to the A2 use cases, this A3 sequence introduces the Worktree Orchestration Engine, External Integration Layer, and DAG Tracker as explicit concrete subsystems supporting cross-device continuity.

8.2 Use Case 2: Pull and Resume a Synced Worktree Session

Use Case Summary: A developer asks Gemini CLI to resume a previously synced task by running `-w pull <taskID>`. The Terminal Interface loads context from the Config System and submits the pull request to the Orchestration Engine. The Orchestration Engine delegates the request to the Worktree Orchestration Engine, which reads the tracker through the External Integration Layer. The External Integration Layer retrieves the task state from the DAG Tracker and returns the tracker information, including the branch reference, context reference, and task state. Using this information, the Worktree Orchestration Engine restores the worktree through the Tool Execution Framework by pulling the task branch from the Git Remote. It then downloads the compressed context through the External Integration Layer, restores that context locally, hydrates the environment, and returns a resume summary to the Orchestration Engine. The Terminal Interface displays the result to the user.

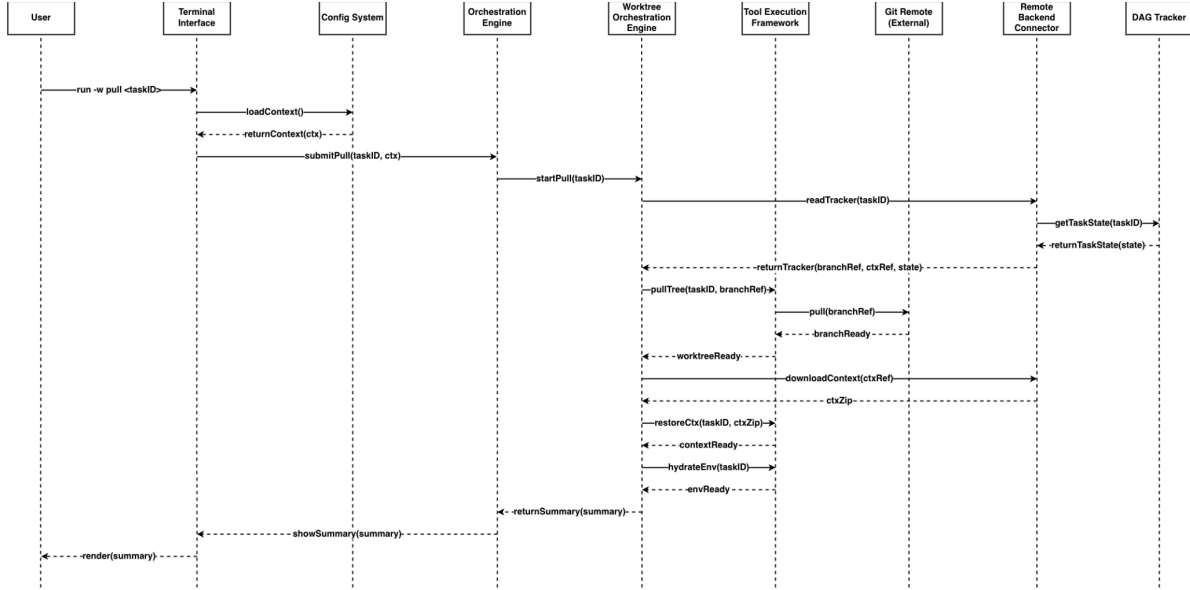


Figure 4: *Sequence Diagram for Use Case #2 (Pull and Resume Synced Worktree Session)*

Explanation: This use case shows the complementary A3 workflow for restoring a task on another device. The Terminal Interface receives the pull command, loads project context from the Config System, and forwards the request to the Orchestration Engine. The Orchestration Engine delegates execution to the Worktree Orchestration Engine, which first retrieves the remote tracker information through the External Integration Layer. The tracker data includes the branch reference, compressed context reference, and task state needed to reconstruct the session. Using the returned branch reference, the Tool Execution Framework restores the task worktree from the Git Remote. After that, the compressed context is downloaded and restored locally, and the environment is hydrated so the task can continue in a usable state. Finally, the Worktree Orchestration Engine returns a summary through the Orchestration Engine to the Terminal Interface. Compared to the A2 use cases, this A3 sequence shifts the focus from one-time code generation and command execution toward persistent state recovery, cross-device work continuity, and coordinated restoration across the new architectural subsystems.

9 Conclusion

This report proposed remote session syncing for Git worktrees as a major enhancement to Gemini CLI. The feature addresses a real weakness in the current system: although Gemini CLI now supports isolated worktree-based task execution, that state is still tied too closely to one device and one active session. Extending worktrees with remote synchronization would let developers resume tasks across devices while preserving task state, compressed context, and workspace isolation. After comparing Localized Worktrees with Extended Remote Capabilities and Containerized Worktree Isolation, we selected the localized worktree approach as the better architectural fit. It provides the best balance between maintainability, evolvability, testability, performance, and usability because it extends existing Git-native workflows instead of introducing a much heavier container management layer. While the enhancement does introduce new security, consistency, and restore-time risks, these can be addressed through careful tracker validation, backend protection, and targeted integration and end-to-end testing. Overall, this enhancement is both practical and well aligned with Gemini CLI's current architecture, and it creates a solid foundation for future multi-device and distributed task workflows.

10 Lessons Learned

In fact, the preparation of this deliverable proved that coming up with architectural improvements is more difficult compared to just presenting an existing architecture. The main difficulty of the group when preparing A1 and A2 was getting a good grasp of the structure of the Gemini CLI and the relationships between its subsystems. The problem for A3, on the other hand, involved making a future-oriented decision that would be realistic based on the current architecture framework. This means that, aside from simply discussing the components of the software architecture, the group also had to take into account other factors such as trade-offs, fit, and the impact of the new functionality on subsystem tasks and quality attributes.

A third thing learned was the significance of maintaining consistency in the definition of an enhancement throughout the report. Given that A3 ties together the description of features, approaches to implementations, SAAM, effects on the architecture, testing, and use cases, even subtle variations in language could lead to contradictions. In this case, this consideration was crucial since the chosen enhancement incorporates several concepts at once, namely Git worktrees, compressed context, DAG-based task state, remote recovery, and multi-device continuity. It was essential to maintain consistency between these concepts to ensure that every section describes the same enhancement rather than somewhat similar enhancements. The importance of such consistency can be seen in the AI report, which emphasizes that it was one of the most difficult aspects of this deliverable.

It was also understood by the team that architectural advancement may lie in leveraging existing strengths as opposed to relying on completely new technologies. The better option here was not necessarily the more complex and sophisticated solution either. Containerized isolation provided for more powerful isolation while localized worktree proved to be the better fit due to better compatibility with existing native Git workflows and architectural pathways. Thus, the group was reminded once again that, when choosing the best software architecture, value, complexity, and architecture should be properly balanced.

Last, this assignment taught us the significance of considering quality attributes at an early stage of suggesting a particular feature. Not only did the suggested improvement need to seem useful for the end users; it also had to be considered on the basis of maintainability, usability, reliability, performance, security, and evolvability. During such an analysis, it became evident that architectural decisions mean choosing certain trade-offs. An architectural decision may enhance one attribute at the cost of another. Remote worktrees, which will be the subject of our discussion during this A3, will increase usability and continuity, but they'll also raise some questions on security, consistency, latency in restoring data, and testing.

11 References

- [1] Gemini CLI. (2026). *Git Worktrees (experimental)*. Gemini CLI. <https://geminicli.com/docs/cli/git-worktrees/>
- [2] Gemini CLI. (2026). *CLI Reference*. Gemini CLI. <https://geminicli.com/docs/cli/cli-reference/>
- [3] Git. (2026). *git-worktree*. Git SCM. <https://git-scm.com/docs/git-worktree>
- [4] Git. (2026). *git-sparse-checkout*. Git SCM. <https://git-scm.com/docs/git-sparse-checkout>

- [5] Google. (2026). *[Epic] Gemini CLI Project Framework: Decoupling Intent (Plans) from State (Trackers)* #22655. GitHub. <https://github.com/google-gemini/gemini-cli/issues/22655>
- [6] Google. (2026). *Git Worktree Isolation & Lifecycle Management* #22658. GitHub. <https://github.com/google-gemini/gemini-cli/issues/22658>
- [7] Google. (2026). *Git worktree support for isolated parallel sessions* #22945. GitHub. <https://github.com/google-gemini/gemini-cli/issues/22945>
- [8] Kazman, R., Klein, M., & Clements, P. (2000). ATAM: Method for architecture evaluation. *Carnegie Mellon Software Engineering Institute*.
- [9] Kazman, R., Bass, L., Abowd, G., & Webb, M. (1994). SAAM: A method for analyzing the properties of software architectures. *Proceedings of the 16th International Conference on Software Engineering*, 81–90.
- [10] Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.
- [11] Docker. (2026). Docker overview. Docker Docs. <https://docs.docker.com/get-started/docker-overview/>
- [12] Docker. (2026). Use bind mounts. Docker Docs. <https://docs.docker.com/engine/storage/bind-mounts/>
- [13] Docker. (2026). Dockerfile reference. Docker Docs. <https://docs.docker.com/reference/dockerfile/>
- [14] Docker. (2026). Best practices for writing Dockerfiles. Docker Docs. <https://docs.docker.com/build/building/best-practices/>
- [15] Linux man-pages. (2026). unix(7) UNIX domain sockets. <https://man7.org/linux/man-pages/man7/unix.7.html>

12 AI Report

1) AI Member Profile and Selection Process

We used **OpenAI ChatGPT (GPT-5.2)**. The hardest part of A3 wasn't generating ideas but keeping the report internally consistent as the enhancement carried through every section. Small wording changes could easily create contradictions across the feature overview, SAAM analysis, architecture impact, risks, testing, and use cases. We also considered Google Gemini and Anthropic Claude. Gemini was relevant to the topic, but ChatGPT worked better for fast iteration, terminology control, and rubric-driven edits, making it a better fit as a constrained review assistant rather than a source of original architectural claims.

2) Tasks Assigned to the AI Teammate

We delegated repetitive, structure-heavy tasks where humans tend to miss small inconsistencies after multiple edits: sequence diagram flow checks (missing handoffs, label inconsistencies, wrong subsystem responsibilities), naming consistency scans across sections and diagrams, readability tightening for

diagram labels, rubric alignment checks, and testing/risk writeup support. All final decisions stayed with humans. We didn't rely on AI for unsupported facts about Gemini CLI internals, and any claim kept in the report had to be justified by course material, repository context, or our own reasoning.

3) Interaction Protocol and Prompting Strategy

One member drove prompts to keep terminology stable across revisions. The team reviewed outputs and flagged follow-up checks when wording felt off or inconsistent. We locked down a canonical vocabulary first, then worked through one section at a time, starting with the feature overview, then implementation approaches, SAAM, architecture impact, risks, testing, and finally the sequence diagrams. AI was required to stay at the architecture level and avoid inventing subsystems not already in the report.

Example prompt: *"Review this A3 sequence diagram for the remote worktree push workflow. Only use these lifelines: Terminal Interface, Config System, Orchestration Engine, Worktree Orchestration Engine, Tool Execution Framework, Git Remote, External Integration Layer, and DAG Tracker. Check whether the ordering matches the written report, keep labels short, and make sure the tracker is written only after the branch and compressed context already exist remotely."*

Early prompts were too open and sometimes pulled in extra components or old terminology. Enforcing a fixed subsystem list and banning invented components fixed that.

4) Validation and Quality Control Procedures

Each suggestion had to pass four checks. First, a mapping check: every lifeline and label had to map to a component already described in the report. Second, a responsibility check: Terminal Interface handles user interaction, Orchestration Engine coordinates the workflow, Worktree Orchestration Engine handles push/pull lifecycle logic, Tool Execution Framework handles execution-side operations, External Integration Layer handles remote communication, and DAG Tracker represents persistent task state. Third, a flow check: push order stayed "push branch, upload context, write tracker" and pull order stayed "fetch tracker, restore worktree, restore context, hydrate environment." Fourth, a section consistency check: if output drifted back toward A2 language or old subsystem sets, it was cut. At least one teammate who didn't make the original request reviewed every revised section for unsupported claims or naming drift.

5) Quantitative Contribution to Final Deliverable

Our honest estimate is that ChatGPT contributed about 20 to 30 percent of the overall A3 effort. The biggest contribution came in sequence diagram refinement, naming consistency, wording cleanup, and testing-plan structure. Moderate contribution came in improving explanations for implementation approaches and use cases after the team had already developed the core ideas. The lowest contribution was in choosing the enhancement, comparing alternatives, identifying stakeholder priorities, and making final architectural calls, all of which stayed human.

6) Reflection on Human-AI Team Dynamics

AI saved real time once the report had a clear direction. It caught places where diagrams and text stopped matching, and flagged sections that still read like A2. That said, it also created extra work. Outputs sometimes sounded right while quietly drifting back to old terminology or adding details we hadn't justified. The fix was tighter prompts and a firm rule: if it can't be traced back to the report's chosen architecture, it's out. The takeaway is simple: AI works well as a fast reviewer. It doesn't replace judgment about what's actually correct.